C++ONLINE

AMLAL EL MAHROUSS

OPEN CONTENT:

THE TPROC LIBRARY

TEXT PROCESSING IN C++

2026

The TProc Library

amlal@nekernel.org

Presentations

amlal@nekernel.org

AMLAL EL MAHROUSS

- C++ Developer.
- Does FOSS for a while.
- Loves C++ and Systems.

The Design Rationale

amlal@nekernel.org

Design Rationale:

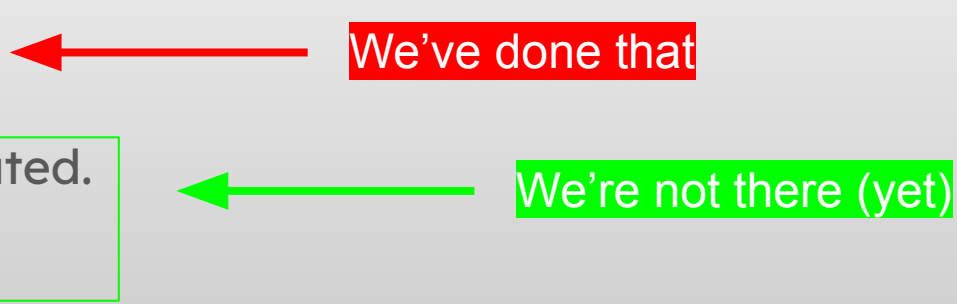
What we aim for:

- Modular-first design.
- Examples-driven.
- Use cases clearly annotated.
- Ease of development.

Design Rationale:

What we aim for:

- Modular-first design.
- Examples-driven.
- Use cases clearly annotated.
- Ease of development.



Design Rationale:

What we aim for:

- Modular-first design.
- Examples-driven.
- Use cases clearly annotated.
- Ease of development.

Design Rationale

```
int main()
{
    auto root     = tproc::crope("The Quick Brown Fox Jumps Over The Lazy Dog");
    auto new_elem = std::make_unique<tproc::crope>(", and Jumps again");
    auto new_elem_2 = std::make_unique<tproc::crope>(", and then Jumps down.");

    boost::asio::io_context ioc{1};
    auto                    spawn_strand = boost::asio::make_strand(ioc);

    boost::asio::co_spawn(spawn_strand, [&new_elem, &new_elem_2, &rope]() ->
boost::asio::awaitable<void> {
        root.concat(new_elem.get());
        new_elem->concat(new_elem_2.get());

        co_return;
    }, boost::asio::detached);

    ocl::asio::run<[]() { (void)0; }>(ioc);

    std::cout << root << std::endl;
}
```

Examples of TProc

amlal@nekernel.org

```cpp
int main()
{
    auto root     = tproc::crope("The Quick Brown Fox Jumps Over The Lazy Dog");
    auto new_elem = std::make_unique<tproc::crope>(", and Jumps again");
    auto new_elem_2 = std::make_unique<tproc::crope>(", and then Jumps down.");

    boost::asio::io_context ioc{1};
    auto                    spawn_strand = boost::asio::make_strand(ioc);

    boost::asio::co_spawn(spawn_strand, [&new_elem, &new_elem_2, &rope]() ->
boost::asio::awaitable<void> {
        root.concat(new_elem.get());
        new_elem->concat(new_elem_2.get());

        co_return;
    }, boost::asio::detached);

    ocl::asio::run<[]() { (void)0; }>(ioc);

    std::cout << root << std::endl;
}
```

```cpp
int main()
{
    auto root     = tproc::crope("The Quick Brown Fox Jumps Over The Lazy Dog");
    auto new_elem = std::make_unique<tproc::crope>(", and Jumps again");
    auto new_elem_2 = std::make_unique<tproc::crope>(", and then Jumps down.");

    boost::asio::io_context ioc{1};
    auto                    spawn_strand = boost::asio::make_strand(ioc);

    boost::asio::co_spawn(spawn_strand, [&new_elem, &new_elem_2, &rope]() ->
boost::asio::awaitable<void> {
        root.concat(new_elem.get());
        new_elem->concat(new_elem_2.get());

        co_return;
    }, boost::asio::detached);

    ocl::asio::run<[]() { (void)0; }>(ioc);

    std::cout << root << std::endl;
}
```

Allocates three cropes.

Concatenates root and new_elem with elem2.

Prints “root” to the stdout.

```cpp
int main()
{
    auto root     = tproc::crope("The Quick Brown Fox Jumps Over The Lazy Dog");
    auto new_elem = std::make_unique<tproc::crope>(", and Jumps again");
    auto new_elem_2 = std::make_unique<tproc::crope>(", and then Jumps down.");

    boost::asio::io_context ioc{1};
    auto                    spawn_strand = boost::asio::make_strand(ioc);

    boost::asio::co_spawn(spawn_strand, [&new_elem, &new_elem_2, &rope]() ->
boost::asio::awaitable<void> {
        root.concat(new_elem.get());
        new_elem->concat(new_elem_2.get());

        co_return;
    }, boost::asio::detached);

    ocl::asio::run<[]() { (void)0; }>(ioc);

    std::cout << root << std::endl;
}
```

Deep dive on the sources

amlal@nekernel.org

Thanks!

Project's repository: https://git.ocl.nekernel.org/tproc

Project's site: https://nekernel.org

Questions?

amlal@nekernel.org